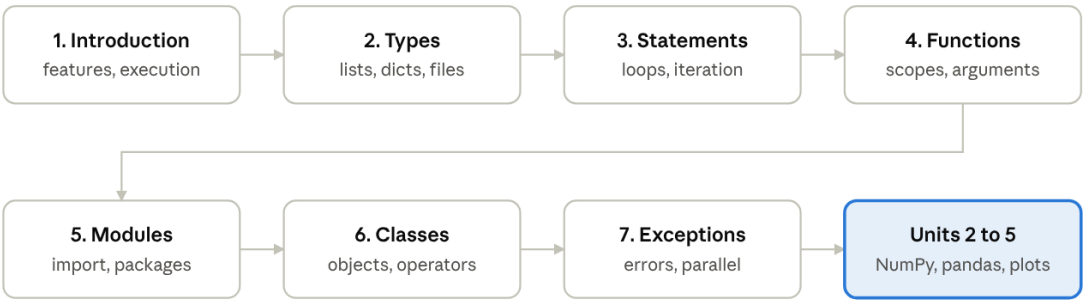
Python Basics and Programming Concepts: Unit 1 Study Guide

Unit overview

Part	Syllabus topics	Why data science needs it
1. Introducing Python	History, features, how code executes	Know where and how your analysis scripts run
2. Types and Operations	Numbers, strings, lists, tuples, dictionaries, files, dynamic typing	Every dataset is held in these structures
3. Statements and Syntax	Assignments, expressions, loops, iterations, comprehensions	Clean and transform records in a few lines
4. Functions	Basics, scopes, arguments, advanced functions	Package analysis steps so they can be reused
5. Modules	Module coding basics, packages, advanced module topics	NumPy and pandas are modules and packages you import
6. Classes and OOP	Classes, operator overloading, class designing	A DataFrame is a class; operators such as + and [] are overloaded
7. Exceptions and Tools	Exception basics, coding details, exception objects, design, parallel system tools	Scripts that survive bad data and use every CPU core

Seven building blocks lead to NumPy, pandas and plotting



Each block uses the ones before it, so study them in this order.

unit roadmap · 7 blocks

The roadmap follows the syllabus order: the first four blocks cover the language itself, and the last three cover how code is organized, modelled and protected from errors.

1. Introducing Python

Python is a high-level, interpreted, general-purpose language created by Guido van Rossum and first released in 1991; it dominates data science because it reads almost like English and has a vast library ecosystem.

1.1 Key features

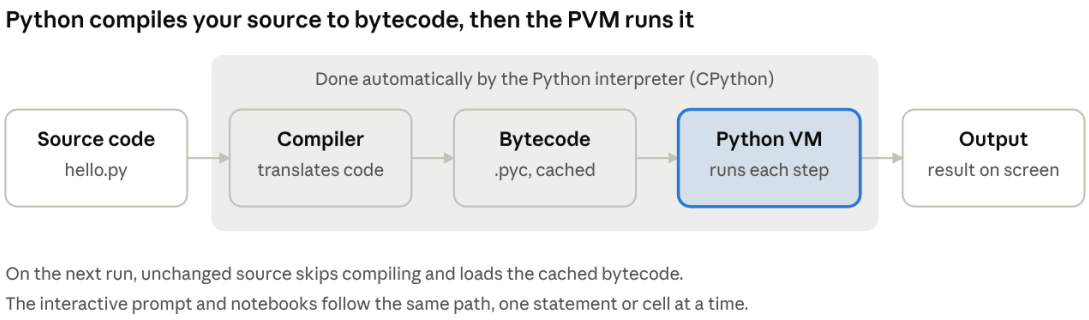
- **High-level:** you work with lists, strings and dictionaries directly, without managing memory.
- **Interpreted:** code runs statement by statement, so you can test ideas instantly in the interactive prompt or a notebook.
- **Dynamically typed:** you never declare variable types; the object carries its type.
- **Multi-paradigm:** supports procedural, object-oriented and functional styles in one program.
- **Open source and portable:** the same script runs on Windows, Linux and macOS.
- **Batteries included:** the standard library covers files, maths, dates, networking, JSON and CSV.

1.2 Why Python for data science

- A rich toolkit: NumPy (arrays), pandas (tables), matplotlib (charts) and scikit-learn (machine learning).
- Short, readable code, so analysts spend time on the problem rather than the syntax.
- Interactive workflows through the REPL and Jupyter notebooks.
- A very large community, so most problems already have a documented answer.

1.3 How Python runs a program

The standard implementation, CPython, first compiles your source file to bytecode, then the Python Virtual Machine (PVM) executes that bytecode one instruction at a time. The compiled bytecode is cached as `.pyc` files inside a `__pycache__` folder so unchanged modules load faster next time.



execution model · 5 stages

Read the diagram left to right: only the three boxes inside the shaded band are done by Python for you.

1.4 Your first program

```
# hello.py
name = "Data Science"
print("Hello,", name)
```

Run it from a terminal with `python hello.py`. You can also type statements one at a time at the interactive prompt (`>>>`), which is ideal for quick experiments.

1.5 Basic syntax rules

- **Indentation defines blocks.** There are no braces; use 4 spaces consistently.
- **A colon (:) opens a block** after `if`, `for`, `while`, `def`, `class` and `try`.
- **Comments** start with #; a string at the top of a function is its docstring.
- **Names are case-sensitive:** `Data` and `data` are different variables.
- **Identifiers** may contain letters, digits and underscores, cannot start with a digit and cannot be keywords such as `if` or `class`.

2. Types and Operations

In Python everything is an object, and every object has a type that decides what operations it supports; the built-in types below hold almost all the data you will analyse.

2.1 Core built-in types at a glance

Type	Example	Mutable?	Ordered?	Typical use
int	`42`	No	n/a	Counts, IDs
float	`3.14`	No	n/a	Measurements
complex	`2 + 3j`	No	n/a	Signal processing
bool	`True`	No	n/a	Conditions, masks
str	`'data'`	No	Yes	Text, labels
list	`[1, 2, 3]`	Yes	Yes	Growing collections
tuple	`(1, 2)`	No	Yes	Fixed records
dict	`{'a': 1}`	Yes	Insertion order	Lookup by key
set	`{1, 2}`	Yes	No	Unique values

A **mutable** object can be changed in place; an **immutable** one can only be replaced by a new object.

2.2 Numbers and numeric types

Python has three built-in numeric types: `int` (whole numbers of unlimited size), `float` (double-precision decimals) and `complex` (real and imaginary parts). `bool` is a subtype of `int`, where `True == 1` and `False == 0`. The standard library adds `Decimal` (exact decimal arithmetic) and `Fraction` (exact rational numbers).

Operator	Meaning	Example	Result
`+`	Addition	`7 + 2`	9
`-`	Subtraction	`7 - 2`	5
`*`	Multiplication	`7 * 2`	14
`/`	True division	`7 / 2`	3.5
`//`	Floor division	`7 // 2`	3
`%`	Remainder	`7 % 2`	1
`**`	Power	`7 ** 2`	49

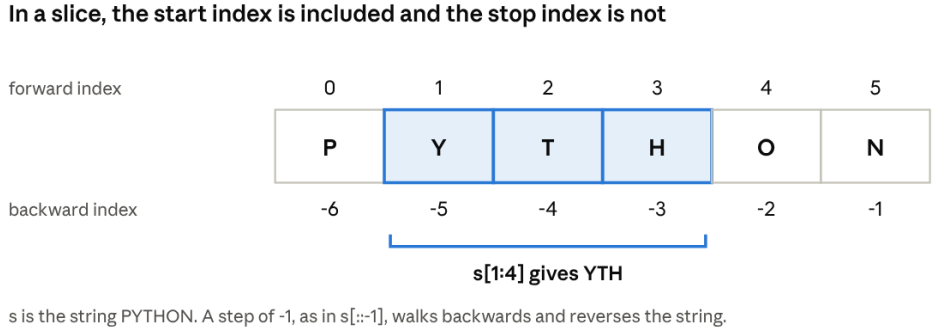
```
print(2 ** 100)            # 1267650600228229401496703205376 (no overflow)
print(0.1 + 0.2)           # 0.30000000000000004 (binary floating point)
print(round(0.1 + 0.2, 2)) # 0.3
print(divmod(17, 5))       # (3, 2) -> quotient and remainder
print(int("25") + float("1.5"))  # 26.5 -> conversion functions
```

When types are mixed, Python converts to the "wider" type (int to float to complex). Floats cannot represent every decimal exactly, so compare them with a tolerance (`math.isclose`) rather than `==`. Useful built-ins are `abs()`, `round()`, `pow()` and `divmod()`; the `math` module adds `sqrt`, `log`, `sin` and constants such as `math.pi`.

2.3 Strings

A string is an **immutable** sequence of characters written in single, double or triple quotes. Because it is a sequence, it supports indexing, slicing, iteration and the `in` operator.

```
s = "Data Science"
print(s[0])        # 'D'      first character
print(s[-1])       # 'e'      last character
print(s[0:4])      # 'Data'   slice: start included, stop excluded
print(s[::-1])     # 'ecneicS ataD' reversed
print(len(s))      # 12
print("Sci" in s)  # True
```



string indexing and slicing · PYTHON

Every character has two indexes, one counted from the front and one from the back; the same rules apply to lists and tuples.

Method	What it does	Example
`upper()` / `lower()`	Change case	`'ab'.upper()` gives `'AB'`
`strip()`	Remove edge whitespace	`' x '.strip()` gives `'x'`
`split(sep)`	Break into a list	`'a,b'.split(',')` gives `['a','b']`
`join(list)`	Glue a list together	`'-'.join(['a','b'])` gives `'a-b'`
`replace(old, new)`	Substitute text	`'cat'.replace('c','b')` gives `'bat'`
`find(sub)`	Index of first match, or -1	`'data'.find('t')` gives 2

Formatting inserts values into text. The modern way is the f-string: `f"Mean = {mean:.2f}"`. Older forms are `"{}".format(x)` and `"%d" % x`. Escape sequences such as `\n` (newline) and `\t` (tab) work in normal strings; prefix with `r` for a raw string (`r"C:\new"`) when backslashes should stay literal.

2.4 Lists

A list is an ordered, **mutable** collection that can hold objects of any type, including other lists. It is Python's workhorse for holding a column or a batch of records.

```
marks = [72, 85, 91]
marks.append(66)           # [72, 85, 91, 66]
marks.insert(1, 80)        # [72, 80, 85, 91, 66]
marks.remove(91)           # [72, 80, 85, 66]
last = marks.pop()         # removes and returns 66
marks.sort(reverse=True)   # [85, 80, 72]
print(marks[0:2])          # [85, 80]
matrix = [[1, 2], [3, 4]]  # nested list
print(matrix[1][0])        # 3
```

Method	Purpose
`append(x)`	Add one item at the end
`extend(iterable)`	Add every item of another sequence
`insert(i, x)`	Insert before index `i`
`remove(x)`	Delete the first occurrence of `x`
`pop(i)`	Remove and return item at `i` (default last)
`sort()` / `reverse()`	Reorder in place
`index(x)` / `count(x)`	Position of `x` / how many times it occurs

Aliasing warning: `b = a` does not copy a list; both names point to the same object. Use `b = a.copy()` or `b = a[:]` for a shallow copy, and `copy.deepcopy(a)` for nested data.

2.5 Tuples

A tuple is an ordered, **immutable** sequence written with parentheses. Use it for a fixed record, such as a coordinate or a row you must not modify, and as a dictionary key (tuples are hashable, lists are not).

```
point = (3, 4)
x, y = point               # unpacking -> x = 3, y = 4
single = (5,)              # the comma makes it a tuple; (5) is just an int
print(point + (5,))        # (3, 4, 5) -> builds a new tuple
point[0] = 9               # TypeError: tuples do not support item assignment
```

Functions that "return several values" actually return one tuple, which the caller unpacks.

2.6 Dictionaries

A dictionary stores **key: value** pairs and finds a value by its key in near-constant time. Keys must be unique and immutable (strings, numbers, tuples); values can be anything. Since Python 3.7 a dictionary remembers insertion order.

```
student = {"name": "Asha", "age": 20, "marks": [72, 85]}
print(student["name"])           # 'Asha'
print(student.get("city", "NA")) # 'NA' -> safe lookup with default
student["age"] = 21              # update
student["city"] = "Kochi"        # add
del student["marks"]             # delete
for key, value in student.items():
    print(key, "->", value)
```

Method	Purpose
`keys()` / `values()` / `items()`	Views of keys, values, or (key, value) pairs
`get(k, default)`	Lookup that never raises `KeyError`
`update(other)`	Merge another dictionary in
`pop(k)`	Remove a key and return its value
`setdefault(k, v)`	Insert `k` with `v` only if it is missing

Dictionaries mirror JSON records and become the rows and columns of a pandas DataFrame later in the course.

2.7 Files

The built-in `open()` returns a file object connected to a file on disk. Always use the `with` statement, which closes the file automatically even if an error occurs.

Mode	Meaning
`'r'`	Read text (default); error if the file is missing
`'w'`	Write text; creates the file or erases an existing

Mode	Meaning
	one
`'a'`	Append to the end of the file
`'r+'`	Read and write
`'b'`	Add to any mode for binary data, such as `'rb'` or `'wb'`

```
with open("scores.txt", "w") as f:
    f.write("Asha,72\n")
    f.write("Ravi,85\n")

with open("scores.txt") as f:
    for line in f:                      # a file is iterable, line by line
        name, mark = line.strip().split(",")
        print(name, int(mark))
```

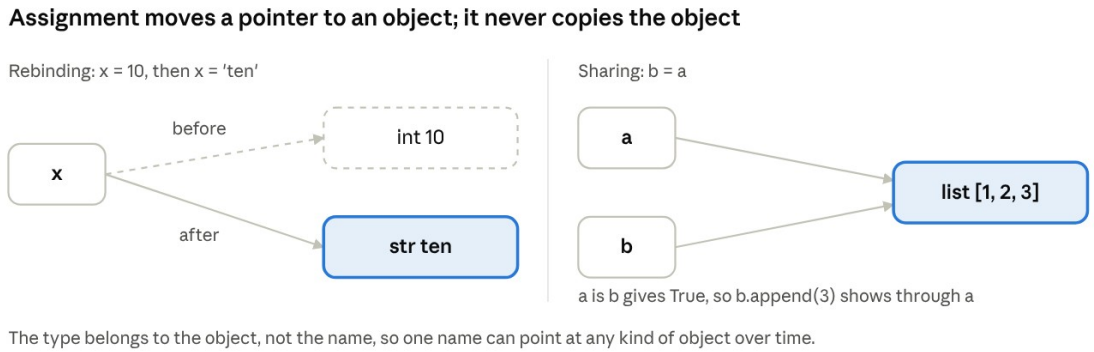
The main read methods are `read()` (whole file as one string), `readline()` (next line) and `readlines()` (list of all lines). Iterating over the file object is the most memory-friendly choice for big files.

2.8 Dynamic typing

In Python a variable is only a **name that refers to an object**. The type lives in the object, not in the name, so the same name can refer to an int now and a string later. Assignment never copies data; it just points the name at an object.

```
x = 10            # x refers to an int object
x = "ten"         # x now refers to a str object; the int is discarded

a = [1, 2]
b = a             # both names refer to the same list
b.append(3)
print(a)          # [1, 2, 3] -> change is visible through both names
print(a is b)     # True  -> same object
print(a == [1, 2, 3])  # True -> equal values
print(type(a), isinstance(a, list))
```



names and objects · rebinding and sharing

The dashed arrow shows the old link that is dropped when a name is rebound; once nothing points to the int, Python frees it.

- **Reference counting:** each object tracks how many names point to it; at zero it is freed by garbage collection.
- **`is` versus `==`:** `is` asks whether two names refer to the *same object*; `==` asks whether the *values* are equal.
- **Consequence:** shared references to mutable objects can cause surprising side effects, while immutable objects (numbers, strings, tuples) are safe to share.
- **Type checking happens at run time:** `"a" + 1` raises `TypeError` only when that line runs.

3. Statements and Syntax

A Python program is a sequence of statements; assignments store results, expressions compute values, and compound statements (conditions and loops) control which statements run and how often.

3.1 Syntax rules for statements

- One statement per line is the norm; a line ends the statement.
- To continue a long statement, wrap it in brackets `( )`, `[ ]`, `{ }` or end the line with a backslash `\`.
- Compound statements (`if`, `for`, `while`, `def`, `class`, `try`, `with`) end their header with a colon and indent the block beneath.
- All lines in one block must share the same indentation; a smaller indent ends the block.

```
total = (10 + 20 +
         30 + 40)        # brackets allow a statement to span lines

if total > 50:
    print("large")        # indented -> belongs to the if block
print("done")             # not indented -> always runs
```

3.2 Assignments

An assignment binds a name to an object. Python offers several assignment forms.

Form	Example	Effect
Basic	`x = 5`	Bind `x` to 5
Chained	`a = b = 0`	Both names refer to the same object
Tuple unpacking	`a, b = 1, 2`	Match values to names by position
Swap	`a, b = b, a`	Exchange values without a temporary
Starred	`first, *rest = [1, 2, 3, 4]`	`rest` becomes `[2, 3, 4]`

Form	Example	Effect
Augmented	`count += 1`	Shorthand for `count = count + 1`

3.3 Expressions

An **expression** produces a value (`3 * 4`, `len(s)`, `x > 2`); a **statement** performs an action (`x = 3`, `print(x)`). An expression on its own line, such as a function call, is an *expression statement*.

- **Precedence** (high to low): `**`, unary `-`, `* / // %`, `+ -`, comparisons, `not`, `and`, `or`. Use brackets when in doubt.
- **Comparison chaining:** `0 < x < 10` works as in mathematics.
- **Short-circuit logic:** in `a and b`, `b` is evaluated only if `a` is true; in `a or b`, only if `a` is false.
- **Conditional expression:** `label = "pass" if mark >= 40 else "fail"`.

3.4 Conditional statements

```
mark = 72
if mark >= 75:
    grade = "A"
elif mark >= 60:
    grade = "B"
else:
    grade = "C"
print(grade)    # B
```

Python tests conditions in order and runs the first block whose test is true. Values such as `0`, `""`, `[]`, `{}` and `None` count as false; everything else is true.

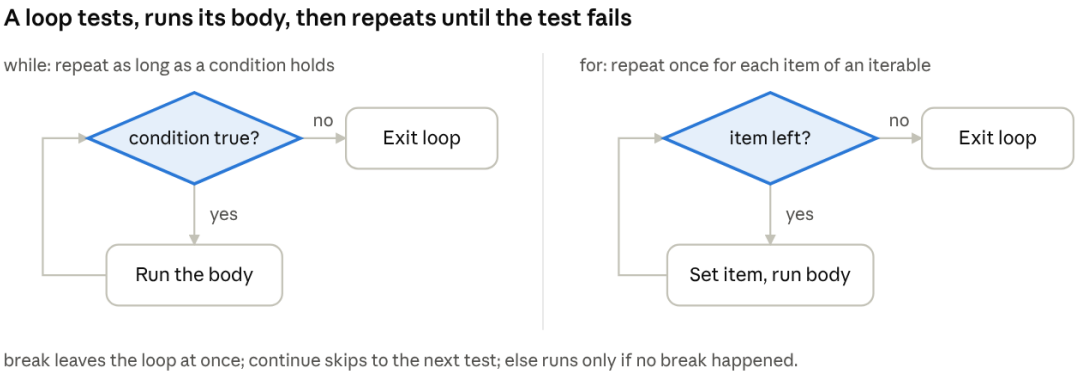
3.5 Loops

A loop repeats a block. Use `for` when you know what to iterate over and `while` when you repeat until a condition changes.

```
for i in range(1, 6):          # 1, 2, 3, 4, 5
    if i == 4:
        continue               # skip the rest of this pass
    print(i)

n = 3
while n > 0:
    print(n)
    n -= 1                     # without this the loop never ends

for x in [2, 4, 6]:
    if x == 5:
        break                  # leave the loop immediately
else:
    print("no break occurred")  # runs only if the loop was not broken
```



while and for loop flow · 2 flowcharts

Both loops share the same cycle; the difference is what is tested, a condition for `while` and the supply of items for `for`.

Statement	Role
`break`	Exit the nearest loop immediately
`continue`	Jump to the next iteration
`pass`	Do nothing; a placeholder for an empty block
`else` on a loop	Runs when the loop finishes without `break`
`range(start, stop, step)`	Sequence of integers; `stop` is excluded

3.6 Iterations

An **iterable** is any object a `for` loop can walk through (list, string, dictionary, file, `range`). When a loop starts, Python calls `iter()` to get an **iterator**, then calls `next()` repeatedly until the iterator raises `StopIteration`.

```
it = iter([10, 20])
print(next(it))    # 10
print(next(it))    # 20
# next(it) would now raise StopIteration

names = ["Asha", "Ravi"]
marks = [72, 85]
for i, name in enumerate(names, start=1):   # index + value
    print(i, name)
for name, mark in zip(names, marks):        # walk two lists together
    print(name, mark)
```

Iterators are lazy: they produce one item at a time, which lets you process files far bigger than memory. Other helpers are `reversed()`, `sorted()`, `sum()`, `min()`, `max()`, `any()` and `all()`.

3.7 Comprehensions

A comprehension builds a new collection from an iterable in a single readable expression. It replaces a loop that appends to a list.

```
squares = [n * n for n in range(1, 6)]             # [1, 4, 9, 16, 25]
evens   = [n for n in range(10) if n % 2 == 0]     # filter
lookup  = {name: len(name) for name in ["Asha", "Ravi"]}   # dict
comprehension
unique  = {m // 10 * 10 for m in [72, 75, 81]}     # set comprehension
total   = sum(n * n for n in range(1000))          # generator expression
```

Kind	Brackets	Result
List comprehension	`[ ... ]`	A list built immediately
Dict comprehension	`{k: v ...}`	A dictionary
Set comprehension	`{ ... }`	A set of unique items
Generator expression	`( ... )`	A lazy iterator; computes items on demand

The general pattern is `[expression for item in iterable if condition]`. Keep comprehensions short; switch to a normal loop when the logic needs several steps.

4. Functions

A function is a named, reusable block of code that takes inputs, does work and returns a result; it is the main tool for avoiding repetition and structuring an analysis.

4.1 Function basics

A function is created with `def`, called with parentheses, and hands a value back with `return`. A function without `return` gives back `None`. The first string in the body is the **docstring**, which documents the function.

```
def mean(values):
    """Return the average of a list of numbers."""
    return sum(values) / len(values)

print(mean([72, 85, 91]))   # 82.66666666666667

def min_max(values):
    return min(values), max(values)   # returns one tuple

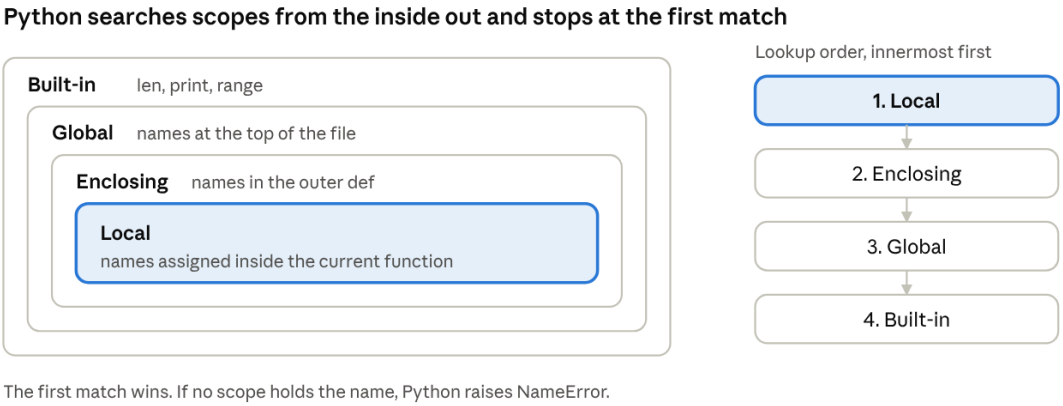
low, high = min_max([4, 9, 2])        # unpacked by the caller
```

`def` is an executable statement: it creates a function object and binds it to a name at run time. Because Python does not check types in advance, `mean` works on a list, a tuple or any iterable of numbers (this is polymorphism).

4.2 Scopes

A **scope** is the region of a program where a name is visible. Python looks a name up in this order, known as **LEGB**:

1. **L**ocal: names assigned inside the current function.
2. **E**nclosing: names in any outer function around the current one.
3. **G**lobal: names at the top level of the module (file).
4. **B**uilt-in: names Python supplies, such as `len`, `print` and `range`.



LEGB scope rule · 4 nested scopes

The nested boxes show where each scope lives; the numbered list on the right is the order in which Python checks them.

```
x = "global"

def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print(x)        # local
    inner()
    print(x)            # enclosing

outer()
print(x)                # global

count = 0
def bump():
    global count        # allow assignment to the module-level name
    count += 1

def counter():
    n = 0
    def step():
        nonlocal n      # allow assignment to the enclosing name
        n += 1
```

```
        return n
    return step
```

Assigning to a name inside a function makes it local by default. Use `global` or `nonlocal` only when you really must change an outer variable; returning values is cleaner.

4.3 Arguments

Arguments are passed by **assignment**: the parameter name is bound to the same object the caller passed. Mutating a list inside a function therefore changes the caller's list, while rebinding the name does not.

Kind	Syntax	Example call
Positional	`def f(a, b)`	`f(1, 2)`
Keyword	`def f(a, b)`	`f(b=2, a=1)`
Default	`def f(a, b=10)`	`f(1)` uses `b = 10`
Variable positional	`def f(*args)`	`f(1, 2, 3)` gives the tuple `(1, 2, 3)`
Variable keyword	`def f(**kwargs)`	`f(x=1, y=2)` gives the dict `{'x': 1, 'y': 2}`
Keyword-only	`def f(a, *, b)`	`f(1, b=2)`
Unpacking at call	`f(*list, **dict)`	`f(*[1, 2])`, `f(**{'a': 1})`

```
def describe(name, age=18, *hobbies, **extra):
    print(name, age, hobbies, extra)

describe("Asha", 20, "chess", "music", city="Kochi")
# Asha 20 ('chess', 'music') {'city': 'Kochi'}

def add_item(item, bucket=[]):      # trap: the default list is created once
    bucket.append(item)
    return bucket

def add_item_safe(item, bucket=None):   # correct pattern
    if bucket is None:
        bucket = []
    bucket.append(item)
    return bucket
```

Rule: never use a mutable object (list, dictionary) as a default value, because it is shared by every call.

4.4 Advanced functions

Functions are **first-class objects**: they can be stored in variables, put in lists and passed to or returned from other functions.

- **lambda:** a small anonymous function written as one expression: `square = lambda x: x * x`.

- **map, filter, reduce:** apply a function across data. `list(map(len, ["ab", "cde"]))` gives `[2, 3]`; `list(filter(lambda n: n > 2, [1, 3, 5]))` gives `[3, 5]`; `functools.reduce` folds a sequence into one value.
- **Recursion:** a function that calls itself, needing a base case to stop.
- **Closures:** an inner function that remembers variables of the enclosing function (the `counter` example above).
- **Generators:** a function using `yield` produces values lazily, one per request, and keeps its state between requests.
- **Decorators:** a function that wraps another function to add behaviour without editing it.
- **Annotations:** optional hints such as `def f(x: int) -> float:` that document intent; Python does not enforce them.

```
def factorial(n):
    return 1 if n <= 1 else n * factorial(n - 1)   # recursion

def countdown(n):
    while n > 0:
        yield n            # generator: pauses here, resumes on next()
        n -= 1
print(list(countdown(3)))  # [3, 2, 1]

import time
def timer(func):           # a decorator
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print(func.__name__, "took", time.time() - start, "seconds")
        return result
    return wrapper

@timer                     # same as: slow_sum = timer(slow_sum)
def slow_sum(n):
    return sum(range(n))
```

5. Modules

A module is simply a Python file whose names (functions, classes, variables) can be reused in other files; modules and packages are how Python code, including NumPy and pandas, is organised and shared.

5.1 Module coding basics

Any file ending in `.py` is a module. Importing it runs the file once and gives you a **module object** whose attributes are the names defined inside it.

```
# stats_tools.py  (the module)
PI = 3.14159

def mean(values):
    return sum(values) / len(values)

def variance(values):
```

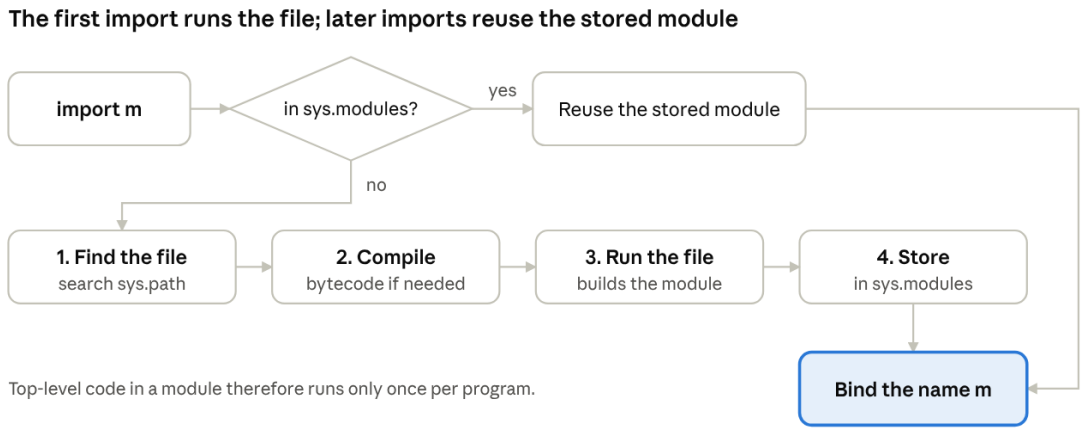
```
    m = mean(values)
    return sum((v - m) ** 2 for v in values) / len(values)
```

```
# analysis.py  (uses the module)
import stats_tools                          # use as stats_tools.mean(...)
import stats_tools as st                    # alias: st.mean(...)
from stats_tools import mean, variance      # bring names in directly
from stats_tools import *                   # everything public; avoid in real
code

print(st.mean([2, 4, 6]))   # 4.0
```

Form	Names you type afterwards	Advice
`import m`	`m.name`	Clear and safe; the default choice
`import m as x`	`x.name`	Standard for long names, such as `import numpy as np`
`from m import a, b`	`a`, `b`	Good for a few frequently used names
`from m import *`	all public names	Risky: can silently overwrite your own names

When Python meets `import m` it performs three steps: **find** the file, **compile** it to bytecode if needed, and **run** it to build the module object. The module is then stored in `sys.modules`, so a second `import m` reuses it instead of running the file again.



import process · 1 decision, 4 steps

The decision at the top is the module cache: a module already in `sys.modules` skips the four steps entirely.

Python searches for modules in this order, listed in `sys.path`: the folder of the running script, any folders in the `PYTHONPATH` environment variable, the standard library, and installed third-party packages in `site-packages`.

5.2 Module packages

A **package** is a folder of modules, imported with dotted names. A folder containing an `__init__.py` file is a regular package; the file runs on import and can expose a friendlier interface.

```
shop/                       <- package
    __init__.py
    inventory.py            <- module
    billing/                <- sub-package
        __init__.py
        tax.py
        invoice.py
```

```
import shop.inventory
from shop.billing import tax
from shop.billing.invoice import make_invoice

# inside shop/billing/invoice.py, a relative import of a sibling module:
from . import tax            # . = this package
from .. import inventory     # .. = parent package
```

Third-party packages are installed with `pip install pandas` and then imported like any other package. This is exactly how `import numpy as np` and `import pandas as pd` work.

5.3 Advanced module topics

- **`__name__` and the main guard.** When a file is run directly its `__name__` is `"__main__"`; when imported it is the module's name. Code under `if __name__ == "__main__":` therefore runs only when the file is executed, not when imported, so one file can serve as both a library and a script.
- **Data hiding.** A name starting with an underscore (`_helper`) signals "internal". Defining `__all__ = ["mean", "variance"]` controls exactly which names `from m import *` exports.
- **Modules are objects.** `dir(module)` lists their names, `module.__file__` gives the file location, and `module.__doc__` gives the docstring.
- **Reloading and dynamic import.** `importlib.reload(m)` re-runs an already imported module (useful in notebooks), and `importlib.import_module("name")` imports using a name held in a string.
- **Circular imports.** If module A imports B while B imports A, one of them may be only partly built. Fix it by moving shared code to a third module or by importing inside a function.
- **Namespaces.** Each module has its own global namespace, so a variable named `x` in two modules never clashes.

```
# stats_tools.py
def mean(values):
    return sum(values) / len(values)
```

```
if __name__ == "__main__":      # runs only for: python stats_tools.py
    print("self-test:", mean([1, 2, 3]))
```

6. Classes and OOP

A class is a blueprint that bundles data (attributes) and behaviour (methods) into one unit; every list, dictionary, NumPy array and pandas DataFrame you use is an instance of a class.

6.1 Class basics

A class defines a new type. Calling the class creates an **instance** (an object). Each instance has its own attributes, while methods are shared by all instances.

```
class Student:
    school = "JNTUGV"                # class attribute: shared by all
instances

    def __init__(self, name, marks):  # constructor: runs when an object is
created
        self.name = name             # instance attributes: unique to each
object
        self.marks = marks

    def average(self):               # method: first parameter is always
self
        return sum(self.marks) / len(self.marks)

    def __str__(self):               # text shown by print()
        return f"{self.name}: {self.average():.1f}"

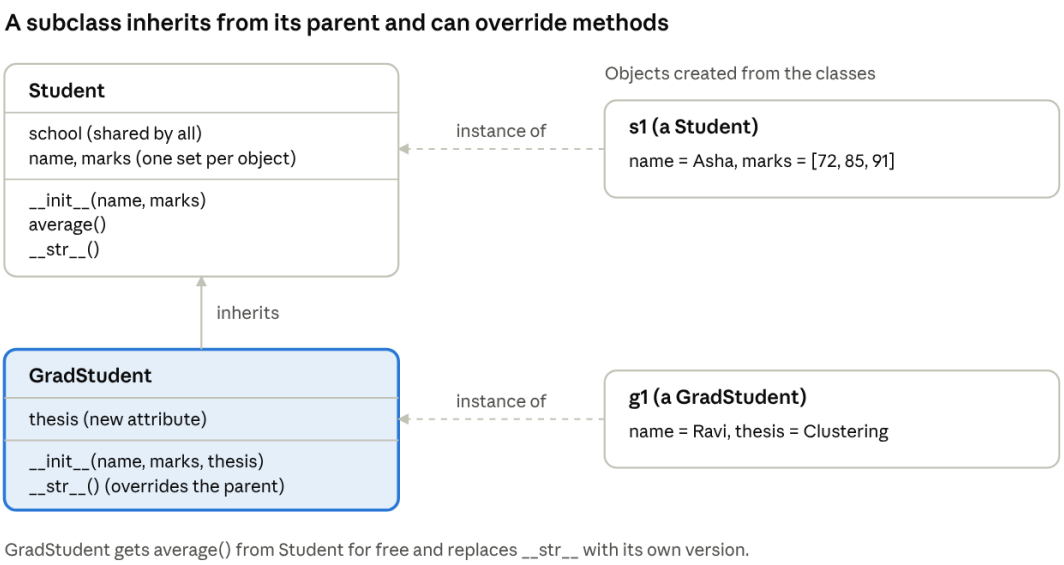
s1 = Student("Asha", [72, 85, 91])
print(s1)                # Asha: 82.7
print(s1.average())      # 82.666...
print(Student.school)    # JNTUGV
```

- **`self`** refers to the instance on which the method was called.
- **`__init__`** is the initializer that sets up a new object's attributes.
- **Class attribute versus instance attribute:** a class attribute is shared; `self.x = ...` creates one attribute per object.

Inheritance lets a new class reuse and extend an existing one. The new class is the *subclass* (child), the existing one the *superclass* (parent).

```
class GradStudent(Student):                 # GradStudent is-a Student
    def __init__(self, name, marks, thesis):
        super().__init__(name, marks)       # reuse the parent's setup
        self.thesis = thesis

    def __str__(self):                      # override the parent's method
        return f"{super().__str__()} | thesis: {self.thesis}"
```



class inheritance · 2 classes, 2 objects

Solid arrows join classes and dashed arrows join an object to its class; method lookup starts at the object's own class and then climbs to the parent.

The four pillars of object-oriented programming are listed below.

Pillar	Meaning	Python example
Encapsulation	Bundle data with the methods that use it, and hide internals	`_balance`, `__secret` naming
Inheritance	Reuse and extend a class	`class Child(Parent)`
Polymorphism	Same call, different behaviour per class	`len(x)` works on lists, strings, dicts
Abstraction	Expose a simple interface, hide the detail	A `fit()` method that hides the algorithm

6.2 Operator overloading

Operators such as `+`, `==`, `[]` and `len()` call special methods whose names begin and end with double underscores ("dunder" methods). By defining them, your own class can work with Python's operators.

Expression	Method called	Purpose
`a + b`	`__add__`	Addition
`a - b`	`__sub__`	Subtraction
`a * b`	`__mul__`	Multiplication

Expression	Method called	Purpose
`a == b`	`__eq__`	Equality
`a < b`	`__lt__`	Ordering and sorting
`len(a)`	`__len__`	Length
`a[i]`	`__getitem__`	Indexing and slicing
`x in a`	`__contains__`	Membership test
`for x in a`	`__iter__`	Iteration
`a(args)`	`__call__`	Make an object callable
`print(a)`	`__str__`	Readable text
`repr(a)`	`__repr__`	Unambiguous text for debugging

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, other):          # v1 + v2
        return Vector(self.x + other.x, self.y + other.y)
    def __mul__(self, k):              # v * 3
        return Vector(self.x * k, self.y * k)
    def __eq__(self, other):           # v1 == v2
        return (self.x, self.y) == (other.x, other.y)
    def __len__(self):                 # len(v)
        return 2
    def __repr__(self):
        return f"Vector({self.x}, {self.y})"

v = Vector(1, 2) + Vector(3, 4)
print(v)            # Vector(4, 6)
print(v * 2)        # Vector(8, 12)
```

This is exactly why `array1 + array2` in NumPy adds element by element and `df["col"]` in pandas selects a column: those classes overload the operators.

6.3 Class designing

Good class design decides what each class is responsible for and how classes relate to one another.

- **Is-a versus has-a.** Use *inheritance* when the child is a kind of the parent (a `GradStudent` is a `Student`). Use *composition* when one object contains another (a `Report` has a `Dataset`). Prefer composition when unsure; it is more flexible.
- **Single responsibility.** One class, one job. A class that loads, cleans, plots and emails data should be split into four.
- **Encapsulation by convention.** A leading underscore (`_x`) means "internal"; a double underscore (`__x`) triggers name mangling. Use `@property` to expose a controlled attribute.

- **Polymorphism and duck typing.** If an object has the method you call, it works, whatever its class. Code against behaviour, not type.
- **Method kinds.** An instance method uses `self`; a `@classmethod` uses `cls` and often builds alternative constructors; a `@staticmethod` is a plain function placed inside the class.
- **Multiple inheritance.** A class may have several parents; Python resolves the lookup order with the method resolution order (MRO), visible as `ClassName.__mro__`.

```
class Dataset:
    def __init__(self, rows):
        self._rows = rows

    @property
    def size(self):                         # read-only computed attribute
        return len(self._rows)

    @classmethod
    def from_csv(cls, path):                # alternative constructor
        with open(path) as f:
            return cls([line.strip().split(",") for line in f])

class Cleaner:
    def drop_empty(self, rows):
        return [r for r in rows if all(r)]

class Pipeline:                             # composition: a Pipeline HAS a
Dataset and a Cleaner
    def __init__(self, data):
        self.data = data
        self.cleaner = Cleaner()
    def run(self):
        return self.cleaner.drop_empty(self.data._rows)
```

7. Exceptions and Tools

An exception is an event that interrupts normal execution when something goes wrong; handling exceptions lets a data-processing script recover from bad input instead of crashing, and system tools let it use the whole machine.

7.1 Exception basics

Python stops at the failing line and looks for a handler. If none exists, the default handler prints a **traceback** (the chain of calls that led to the error) and ends the program. A `try` statement lets you catch the error and carry on.

```
try:
    value = int(input("Enter a number: "))
    print(10 / value)
except ValueError:
    print("That was not a number")
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Exception	Typical cause
`SyntaxError`	Invalid code; found before the program runs
`NameError`	Using a variable that was never defined
`TypeError`	Wrong type for an operation, such as `"a" + 1`
`ValueError`	Right type, unsuitable value, such as `int("abc")`
`IndexError`	List index out of range
`KeyError`	Missing dictionary key
`ZeroDivisionError`	Division or modulo by zero
`FileNotFoundError`	Opening a file that does not exist
`AttributeError`	Object has no such attribute or method
`ImportError`	A module or name cannot be imported

7.2 Exception coding details

The full `try` statement has four parts, and each plays a distinct role.

Clause	Runs when	Use it for
`try`	Always first	The risky code
`except`	The `try` block raised a matching exception	Recovery or a clear message
`else`	The `try` block raised **no** exception	Code that should run only on success
`finally`	Always, error or not	Cleanup such as closing a file

↗ try statement flow · 2 decisions, 3 outcomes

Follow the three lanes from top to bottom: no error, an error that is caught, and an error that is not; all three end in the finally block.

```
def read_scores(path):
    try:
        f = open(path)
    except FileNotFoundError as err:
        print("Missing file:", err)
        return []
    except (PermissionError, IsADirectoryError):   # several types in a tuple
        print("Cannot read this path")
        return []
    else:
```

```
        scores = [int(line) for line in f]          # only if open() worked
        return scores
    finally:
        print("read_scores finished")               # runs in every case
```

- **`raise`** triggers an exception yourself: `raise ValueError("age must be positive")`. A bare `raise` inside an `except` block re-raises the current error.
- **`raise NewError from err`** chains a new exception to the original cause.
- **`assert`** checks an assumption while debugging: `assert len(rows) > 0, "no data"` raises `AssertionError` when the test is false.
- **`with`** uses a *context manager* for guaranteed cleanup, which is why `with open(...)` is preferred to a manual `try/finally`.
- **Propagation:** an uncaught exception travels up through the calling functions until a matching handler is found or the program stops.

7.3 Exception objects

Exceptions are instances of classes arranged in an inheritance tree. Every built-in exception descends from `BaseException`; ordinary errors descend from `Exception`. An `except` clause catches the named class **and all its subclasses**, so `except LookupError` catches both `IndexError` and `KeyError`.

↗ exception class hierarchy · 14 classes

Read the tree left to right: `except LookupError` would catch `IndexError` and `KeyError`, while `except Exception` catches everything in the highlighted branch.

```
try:
    {}["missing"]
except KeyError as e:        # e is the exception object
    print(type(e).__name__)  # KeyError
    print(e.args)            # ('missing',)
    print(e)                 # 'missing'
```

You can define your own exception by subclassing `Exception`. Custom classes can carry extra data about the failure.

```
class InvalidRecordError(Exception):
    def __init__(self, row_number, message):
        super().__init__(f"Row {row_number}: {message}")
        self.row_number = row_number

def check(row, n):
    if row.get("age", 0) <= 0:
        raise InvalidRecordError(n, "age must be positive")

try:
    check({"age": -3}, 7)
except InvalidRecordError as e:
    print(e, "| bad row:", e.row_number)   # Row 7: age must be positive |
bad row: 7
```

7.4 Designing with exceptions

- **Catch specific exceptions.** Avoid a bare `except:` or a blanket `except Exception:`; they hide real bugs such as typos.
- **Handle an error where you can fix it.** Let the exception travel up to the layer that knows what to do (retry, skip the row, show a message).
- **Keep `try` blocks small,** wrapping only the line that can fail, so you know which statement raised.
- **EAFP over LBYL.** "Easier to ask forgiveness than permission" (try it and catch the error) is the Python style; "look before you leap" checks conditions first and can be slower or racy.
- **Always release resources** with `finally` or `with`.
- **Use custom exceptions** to express problems in your domain, and group them under one base class so callers can catch them together.
- **Do not use exceptions for ordinary control flow** when a simple `if` is clearer. The exception is `StopIteration`, which Python itself uses to end loops.
- **Log, do not hide.** Record the traceback with the `logging` module so failures can be diagnosed later.

```
import logging

def load_rows(path):
    good, bad = [], []
    with open(path) as f:
        for n, line in enumerate(f, start=1):
            try:
                good.append(float(line))
            except ValueError:                    # specific, small,
recoverable
                bad.append(n)
                logging.warning("Skipped bad row %d", n)
    return good, bad
```

7.5 Parallel system tools

Data jobs are often slow because they wait for disks and networks (**I/O-bound**) or because they crunch numbers (**CPU-bound**). Python's system tools let you talk to the operating system and run work in parallel.

System interfaces. The `sys` module exposes the interpreter (`sys.argv` for command-line arguments, `sys.path`, `sys.exit()`); the `os` module exposes the operating system (`os.getcwd()`, `os.listdir()`, `os.environ`, `os.cpu_count()`); and `subprocess` launches other programs and captures their output.

Parallel tools.

Tool	Module	Best for	Notes
Threads	`threading`, `concurrent.futures.ThreadPoolExecutor`	I/O-bound work (downloads, file reads)	Share memory; in the standard build the Global Interpreter Lock

Tool	Module	Best for	Notes
			(GIL) lets only one thread run Python bytecode at a time
Processes	`multiprocessing`, `concurrent.futures.ProcessPoolExecutor`	CPU-bound work (heavy calculation)	Separate memory and interpreter per process, so they use several cores
Asynchronous I/O	`asyncio`	Thousands of waiting network tasks	One thread, cooperative `async` and `await`

Recent Python releases (3.13 onward) add an optional free-threaded build without the GIL, but the standard build still has it. For CPU-bound data work, the usual choices are multiple processes or libraries such as NumPy that run their inner loops in compiled code.

```
from concurrent.futures import ProcessPoolExecutor
import os

def heavy(n):
    return sum(i * i for i in range(n))

if __name__ == "__main__":                  # required for multiprocessing
on Windows/macOS
    jobs = [10_000_000] * os.cpu_count()
    with ProcessPoolExecutor() as pool:       # one worker process per core
by default
        results = list(pool.map(heavy, jobs)) # runs the jobs in parallel
    print(len(results), "jobs finished")
```

Quick revision

If you remember only one idea per topic, remember the ones in this table.

Topic	One idea to remember
Introducing Python	Source code is compiled to bytecode, then run by the Python Virtual Machine; indentation defines blocks
Numbers	`int`, `float`, `complex`; `/` gives a float, `//` floors, `**` is a power
Strings	Immutable sequences; slice with `s[start:stop:step]`, with stop excluded
Lists	Mutable and ordered; `b = a` shares one object, `a.copy()` makes a new one

Topic	One idea to remember
Tuples	Immutable; used for fixed records, unpacking and dictionary keys
Dictionaries	Key-to-value lookup; keys must be immutable and unique
Files	Use `with open(...)`; modes `r`, `w`, `a`, plus `b` for binary
Dynamic typing	Names refer to objects; the type belongs to the object; `is` compares identity, `==` compares value
Statements	Assignment binds names; `if/elif/else` branches; `for` and `while` repeat
Iterations	Iterable gives an iterator through `iter()`; `next()` ends with `StopIteration`
Comprehensions	`[expr for x in it if cond]` builds a collection in one line
Functions	`def`, `return`, docstring; arguments are passed by object reference
Scopes	Lookup order is Local, Enclosing, Global, Built-in (LEGB)
Arguments	Positional, keyword, default, `*args`, `**kwargs`; never use a mutable default
Advanced functions	Lambda, map, filter, closures, generators (`yield`), decorators (@)
Modules	A `.py` file; `import` finds, compiles and runs it once, then caches it in `sys.modules`
Packages	A folder with `__init__.py`; imported with dotted names
Classes	`__init__` sets up the object; `self` is the instance; subclasses override methods
Operator overloading	`a + b` calls `a.__add__(b)`; `len(a)` calls `a.__len__()`
Exceptions	`try`, `except`, `else`, `finally`; catch specific types; `raise` to signal errors
Parallel tools	Threads for I/O waits, processes for CPU work,

Topic	One idea to remember
	because of the GIL

Likely exam questions

Short answer (2 to 3 marks)

1. List any four features of Python.
2. Differentiate between a list and a tuple.
3. What is dynamic typing? Give an example.
4. Write the output of `[x * x for x in range(4)]`.
5. State the LEGB rule.
6. What is the purpose of `if __name__ == "__main__":`?
7. Differentiate between `is` and `==`.
8. What is the role of the `finally` clause?

Long answer (5 to 10 marks)

1. Explain how a Python program is executed, with a diagram.
2. Describe the core built-in data types of Python with examples of their operations.
3. Explain the different types of function arguments with examples.
4. Explain how modules and packages are created and imported.
5. Explain classes, inheritance and operator overloading with a program.
6. Explain exception handling in Python, covering `try`, `except`, `else`, `finally` and user-defined exceptions.
7. Compare threads, processes and asynchronous I/O for parallel programming in Python.

References

The first two lists are the books prescribed in the syllabus; the third lists the official Python documentation that matches each part of this unit.

Text books

1. Mark Lutz, *Learning Python*, O'Reilly.
2. Mark Lutz, *Programming Python*, O'Reilly.
3. Wes McKinney, *Python for Data Analysis*, O'Reilly.

Reference books

1. Martin C. Brown, *Python: The Complete Reference*, McGraw Hill Education.
2. Paul Barry, *Head First Python*, O'Reilly.